📓 Panduan Penyelesaian Latihan & Tugas

Materi : Pengujian dan Integrasi Kode

🎯 Gambaran Umum

Panduan ini membimbing Anda menyelesaikan **3 latihan** dan **1 tugas mandiri** dari materi kuliah secara bertahap, dari nol hingga siap submit.

Estimasi waktu total: 4–6 jam

Aktivitas	Estimasi	Output
Setup environment	30 menit	Project siap
Latihan 1: Refactor Test	45 menit	`tests/User.test.js`
Latihan 2: Pipeline CI	30 menit	`.github/workflows/ci.yml`
Latihan 3: Analisis Kasus	30 menit	Dokumen analisis
Tugas Mandiri	2–3 jam	Repo GitHub lengkap

🔧 BAGIAN 0 — Persiapan Environment (30 menit)

Langkah 0.1 — Verifikasi Tools

Buka terminal, jalankan:

```bash
node --version    # harus v18+
npm --version     # harus 9+
git --version     # harus 2.30+
```

Jika ada yang belum terpasang → install dari:

- Node.js: https://nodejs.org (pilih LTS)
- Git: https://git-scm.com

Langkah 0.2 — Buat Folder Project

```bash
mkdir standarisasi-coding
cd standarisasi-coding
git init
```

Langkah 0.3 — Inisialisasi Node.js Project

bash

```
npm init -y
```

Buka `package.json`, ubah menjadi:

json

```
{
  "name": "standarisasi-coding",
  "version": "1.0.0",
  "description": "Latihan pengujian dan integrasi kode",
  "scripts": {
    "test": "jest",
    "test:coverage": "jest --coverage",
    "lint": "eslint .",
    "format:check": "prettier --check .",
    "coverage:check": "jest --coverage --coverageThreshold='{\"global\":{\"lines\":80,\"branches\":70}}'"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "eslint": "^8.57.0",
    "prettier": "^3.2.0"
  }
}
```

Langkah 0.4 — Install Dependencies

bash

```
npm install
```

Langkah 0.5 — Buat Struktur Folder

bash

```
mkdir -p src tests .github/workflows
```

Langkah 0.6 — Buat `.gitignore`

bash

```
cat > .gitignore << 'EOF'
node_modules/
coverage/
*.log
.env
.DS_Store
EOF
```

Langkah 0.7 — Konfigurasi Jest

Buat `jest.config.js`:

javascript

```
module.exports = {
  testEnvironment: 'node',
  collectCoverageFrom: ['src/**/*.js'],
```

```
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'lcov', 'clover'],
  coverageThreshold: {
    global: {
      branches: 70,
      functions: 70,
      lines: 80,
      statements: 80,
    },
  },
};
```

Langkah 0.8 — Konfigurasi ESLint

Buat `.eslintrc.json`:

json

```
{
  "env": {
    "node": true,
    "es2022": true,
    "jest": true
  },
  "extends": "eslint:recommended",
  "parserOptions": {
    "ecmaVersion": 2022,
    "sourceType": "module"
```

```
  },
  "rules": {
    "no-unused-vars": "warn",
    "no-console": "off"
  }
}
```

Langkah 0.9 — Konfigurasi Prettier

Buat `.prettierrc`:

```
json
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5"
}
```

✅ Checklist Bagian 0

- □ Node.js & Git terpasang
- □ Folder project dibuat & `git init` berhasil
- □ `npm install` sukses
- □ Struktur folder `src/`, `tests/`, `.github/workflows/` ada

- □ File konfigurasi (`.gitignore`, `jest.config.js`, `.eslintrc.json`, `.prettierrc`) dibuat

📝 LATIHAN 1 — Refactor Test (45 menit)

🎯 Tujuan

Memperbaiki test yang tidak standar menjadi test dengan struktur **AAA** dan penamaan `should...when...`.

Langkah 1.1 — Buat Source Code `User.js`

Buat `src/User.js`:

```
javascript
class User {
  constructor(name, email, age) {
    if (!name || name.trim() === '') {
      throw new Error('Name required');
    }
    if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      throw new Error('Invalid email');
    }
    if (age < 0 || age > 150) {
      throw new Error('Invalid age');
    }
    this.name = name;
    this.email = email;
    this.age = age;
  }
```

```
  getName() {
    return this.name;
  }

  getEmail() {
    return this.email;
  }

  getAge() {
    return this.age;
  }
}

module.exports = User;
```

Langkah 1.2 — Buat File Test Baru

Buat `tests/User.test.js`:

```
javascript
const User = require('../src/User');

describe('User', () => {
  describe('when created with valid data', () => {
    let user;
```

```
  beforeEach(() => {
    // Arrange
    user = new User('John', 'john@test.com', 25);
  });

  test('should return correct name', () => {
    expect(user.getName()).toBe('John');
  });

  test('should return correct email', () => {
    expect(user.getEmail()).toBe('john@test.com');
  });

  test('should return correct age', () => {
    expect(user.getAge()).toBe(25);
  });
});

describe('when created with invalid data', () => {
  test('should throw error when name is empty', () => {
    expect(() => new User('', 'a@b.com', 25)).toThrow('Name required');
  });

  test('should throw error when name is only whitespace', () => {
    expect(() => new User('   ', 'a@b.com', 25)).toThrow('Name required');
  });
```

```
    test('should throw error when email is invalid', () => {
      expect(() => new User('John', 'invalid', 25)).toThrow('Invalid email');
    });

    test('should throw error when email is empty', () => {
      expect(() => new User('John', '', 25)).toThrow('Invalid email');
    });

    test('should throw error when age is negative', () => {
      expect(() => new User('John', 'a@b.com', -1)).toThrow('Invalid age');
    });

    test('should throw error when age exceeds 150', () => {
      expect(() => new User('John', 'a@b.com', 200)).toThrow('Invalid age');
    });
  });
});
```

Langkah 1.3 — Jalankan Test

```
bash
npm test
```

Output yang diharapkan:

```
text
PASS  tests/User.test.js
```

```
  User
    when created with valid data
      ✓ should return correct name
      ✓ should return correct email
      ✓ should return correct age
    when created with invalid data
      ✓ should throw error when name is empty
      ✓ should throw error when name is only whitespace
      ✓ should throw error when email is invalid
      ✓ should throw error when email is empty
      ✓ should throw error when age is negative
      ✓ should throw error when age exceeds 150

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
```

Langkah 1.4 — Cek Coverage

```
bash
npm run test:coverage
```

Cek output: Coverage `User.js` harus **100%** karena semua branch teruji.

Langkah 1.5 — Commit

```
bash
```

```
git add .
git commit -m "test(user): add comprehensive unit tests with AAA structure"
```

✅ Checklist Latihan 1

- □ `src/User.js` dibuat
- □ `tests/User.test.js` mengikuti struktur AAA
- □ Penamaan `should...when...` diterapkan
- □ Semua test **passing**
- □ Coverage 100% untuk `User.js`
- □ Commit dengan conventional commit message

🚀 LATIHAN 2 — Pipeline CI (30 menit)

🎯 Tujuan

Membuat pipeline GitHub Actions yang menjalankan lint, test, dan coverage check otomatis.

Langkah 2.1 — Buat Repo di GitHub

1. Buka https://github.com/new
2. Nama: `standarisasi-coding`
3. **Public** (agar Actions gratis unlimited)
4. **Jangan** centang "Initialize with README"
5. Klik **Create repository**

Langkah 2.2 — Hubungkan Local ke GitHub

```bash
git branch -M main
git remote add origin https://github.com/USERNAME/standarisasi-coding.git
git push -u origin main
```

Ganti `USERNAME` dengan username GitHub Anda.

Langkah 2.3 — Buat Workflow CI

Buat `.github/workflows/ci.yml`:

```yaml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Test & Coverage
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
```

```
      run: npm ci

    - name: Run linter
      run: npm run lint

    - name: Check formatting
      run: npm run format:check

    - name: Run tests with coverage
      run: npm test -- --coverage --ci

    - name: Enforce coverage threshold
      run: npm run coverage:check

    - name: Upload coverage artifact
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: coverage-report
        path: coverage/
        retention-days: 7
```

⚠ **Penting:** `npm ci` memerlukan `package-lock.json`. Pastikan sudah ada (dibuat otomatis oleh `npm install`). Jika belum, jalankan `npm install` sekali lalu commit `package-lock.json`.

Langkah 2.4 — Commit & Push

```bash
git add .github/workflows/ci.yml package-lock.json
git commit -m "ci: add GitHub Actions pipeline with lint, test, and coverage"
git push
```

Langkah 2.5 — Lihat Hasil CI

1. Buka repo di GitHub
2. Klik tab **Actions**
3. Klik workflow run terbaru
4. Pastikan semua step **hijau** ✅

Langkah 2.6 — Uji Pipeline Gagal (Opsional tapi Disarankan)

Buat test yang sengaja gagal untuk melihat CI menolak:

```bash
# Tambahkan test gagal sementara
echo "test('should fail intentionally', () => { expect(1).toBe(2); });" >> tests/User.test.js
git add tests/User.test.js
git commit -m "test: add failing test (temporary)"
git push
```

Di GitHub Actions, workflow harus **merah** ❌.

Setelah itu, **hapus** test gagal tersebut dan push lagi:

```bash
git revert HEAD --no-edit
git push
```

✅ Checklist Latihan 2

- □ Repo GitHub dibuat (public)
- □ `.github/workflows/ci.yml` ada
- □ Pipeline hijau ✅ di GitHub Actions
- □ Sudah uji pipeline gagal (opsional)
- □ Commit dengan message konvensional

🧠 LATIHAN 3 — Analisis Kasus (30 menit)

🎯 Tujuan

Menganalisis masalah tim dan merancang roadmap perbaikan.

Langkah 3.1 — Buat Dokumen Analisis

Buat `ANALISIS.md` di root project:

```
markdown
# Analisis Kasus: Tim dengan 15 Developer

## 1. Identifikasi Masalah

Berdasarkan skenario:
- 15 developer, semua commit langsung ke `main`
- Tidak ada CI/CD
- Test coverage 25%
- Bug production setiap minggu

### Masalah Utama:

| # | Masalah | Dampak |
|---|---------|--------|
| 1 | Tidak ada branch strategy | Konflik merge, main sering rusak |
```

```
| 2 | Tidak ada code review | Kode berkualitas rendah masuk |
| 3 | Tidak ada CI/CD | Bug tidak terdeteksi sebelum production |
| 4 | Coverage 25% | Regresi tidak terdeteksi |
| 5 | Tidak ada quality gate | Tidak ada standar minimum kualitas |

### Akar Masalah:

**Kurangnya standarisasi proses integrasi kode** — semua developer bekerja
tanpa kontrak yang jelas tentang bagaimana kode diintegrasikan.
```

2. Standar yang Harus Diterapkan Pertama

Prioritas 1 (Minggu 1-2): Branch Protection + Code Review

- Alasan: Menghentikan pendarahan (stop the bleeding). Tanpa ini, semua perbaikan lain akan sia-sia karena main tetap bisa rusak.
- Aksi: Aktifkan branch protection di `main`, wajib 1 reviewer.

Prioritas 2 (Minggu 3-4): CI Sederhana

- Alasan: Deteksi dini kerusakan build & test.
- Aksi: GitHub Actions minimal — lint + test.

Prioritas 3 (Minggu 5-8): Naikkan Coverage Bertahap

- Alasan: Tidak bisa langsung 80% dari 25% (terlalu besar lompatan).
- Aksi: Target +10% per 2 minggu.

Prioritas 4 (Minggu 9-12): Quality Gate + CD

- Alasan: Setelah budaya terbentuk, otomasi gate penuh.
- Aksi: Coverage threshold di CI, auto-deploy ke staging.

3. Roadmap 3 Bulan

Bulan 1 — Fondasi

- □ Aktifkan branch protection di `main`
- □ Terapkan conventional commits
- □ Wajib PR + 1 reviewer
- □ Setup CI dasar (lint + test)
- □ Sosialisasi standar ke tim

Bulan 2 — Peningkatan

- □ Coverage naik ke 50%
- □ Wajib unit test untuk fitur baru
- □ Setup coverage reporting (Codecov)
- □ Mulai code review checklist
- □ Refactor 5 modul paling kritis dengan test

Bulan 3 — Otomasi Penuh

- □ Coverage naik ke 70%
- □ Quality gate aktif (CI gagal jika coverage turun)
- □ CD ke staging otomatis
- □ Dokumentasi standar lengkap
- □ Retrospective & ukur bug production

4. Metrik Keberhasilan

Metrik	Baseline	Target 3 Bulan
Test coverage	25%	≥70%
Bug production/minggu	~5	≤1
Waktu merge PR	?	<1 hari
Broken main/bulan	?	0
Waktu code review	?	<4 jam

```
text

### Langkah 3.2 — Commit

```bash
git add ANALISIS.md
git commit -m "docs: add case analysis and 3-month improvement roadmap"
git push
```
```

✅ Checklist Latihan 3

- □ Dokumen `ANALISIS.md` berisi 4 bagian (masalah, prioritas, roadmap, metrik)
- □ Justifikasi prioritas jelas
- □ Roadmap terukur per bulan
- □ Commit & push

🎓 TUGAS MANDIRI (2–3 jam)

🎯 Tujuan

Membuat project lengkap dengan **minimal 10 test case AAA + CI + coverage ≥70%**.

Langkah 4.1 — Pilih Domain Project

Pilih **salah satu** tema (atau buat sendiri yang relevan):

Tema	Contoh Fungsi
Shopping Cart	add, remove, calculate total, apply discount, tax
Todo List	add, complete, delete, filter, count
Bank Account	deposit, withdraw, transfer, getBalance
Validator	email, password strength, phone, NIK, URL
String Utils	reverse, palindrome, slugify, camelCase

💡 **Rekomendasi:** pilih **Shopping Cart** karena kaya edge case → mudah mencapai 10+ test.

Langkah 4.2 — Buat Source Code

Contoh `src/ShoppingCart.js`:

```
javascript
class ShoppingCart {
```

```
constructor() {
  this.items = [];
}

addItem(item) {
  if (!item || !item.name || item.price == null) {
    throw new Error('Invalid item');
  }
  if (item.price < 0) {
    throw new Error('Price cannot be negative');
  }
  const existing = this.items.find((i) => i.name === item.name);
  if (existing) {
    existing.quantity += item.quantity || 1;
  } else {
    this.items.push({ ...item, quantity: item.quantity || 1 });
  }
}

removeItem(name) {
  const index = this.items.findIndex((i) => i.name === name);
  if (index === -1) {
    throw new Error('Item not found');
  }
  this.items.splice(index, 1);
}
```

```
  calculateSubtotal() {
    return this.items.reduce(
      (sum, item) => sum + item.price * item.quantity,
      0
    );
  }

  calculateTotal(discountRate = 0, taxRate = 0) {
    if (discountRate < 0 || discountRate > 1) {
      throw new Error('Invalid discount rate');
    }
    if (taxRate < 0 || taxRate > 1) {
      throw new Error('Invalid tax rate');
    }
    const subtotal = this.calculateSubtotal();
    const afterDiscount = subtotal * (1 - discountRate);
    return Math.round(afterDiscount * (1 + taxRate));
  }

  getItemCount() {
    return this.items.reduce((sum, item) => sum + item.quantity, 0);
  }

  clear() {
    this.items = [];
  }
}
```

```
module.exports = ShoppingCart;
```

Langkah 4.3 — Tulis Minimal 10 Test AAA

Buat `tests/ShoppingCart.test.js`:

```
javascript
const ShoppingCart = require('../src/ShoppingCart');

describe('ShoppingCart', () => {
  let cart;

  beforeEach(() => {
    cart = new ShoppingCart();
  });

  // ===== addItem =====
  describe('addItem', () => {
    test('should add item when valid item given', () => {
      // Arrange
      const item = { name: 'Book', price: 50000 };
      // Act
      cart.addItem(item);
      // Assert
      expect(cart.items).toHaveLength(1);
      expect(cart.items[0].name).toBe('Book');
```

```
  });

  test('should increment quantity when same item added twice', () => {
    cart.addItem({ name: 'Book', price: 50000 });
    cart.addItem({ name: 'Book', price: 50000 });
    expect(cart.items[0].quantity).toBe(2);
  });

  test('should throw error when item has no name', () => {
    expect(() => cart.addItem({ price: 1000 })).toThrow('Invalid item');
  });

  test('should throw error when price is negative', () => {
    expect(() => cart.addItem({ name: 'X', price: -1 })).toThrow(
      'Price cannot be negative'
    );
  });
});

// ===== removeItem =====
describe('removeItem', () => {
  test('should remove item when existing item name given', () => {
    cart.addItem({ name: 'Book', price: 50000 });
    cart.removeItem('Book');
    expect(cart.items).toHaveLength(0);
  });
```

```
  test('should throw error when removing non-existent item', () => {
    expect(() => cart.removeItem('Ghost')).toThrow('Item not found');
  });
});

// ===== calculateSubtotal =====
describe('calculateSubtotal', () => {
  test('should return 0 when cart is empty', () => {
    expect(cart.calculateSubtotal()).toBe(0);
  });

  test('should sum price times quantity for all items', () => {
    cart.addItem({ name: 'Book', price: 50000, quantity: 2 });
    cart.addItem({ name: 'Pen', price: 10000 });
    expect(cart.calculateSubtotal()).toBe(110000);
  });
});

// ===== calculateTotal =====
describe('calculateTotal', () => {
  test('should return subtotal when no discount and no tax', () => {
    cart.addItem({ name: 'Book', price: 100000 });
    expect(cart.calculateTotal()).toBe(100000);
  });

  test('should apply discount correctly', () => {
    cart.addItem({ name: 'Book', price: 100000 });
```

```
    expect(cart.calculateTotal(0.1)).toBe(90000);
  });

  test('should apply discount then tax correctly', () => {
    cart.addItem({ name: 'Book', price: 100000 });
    // 100000 * 0.9 = 90000, * 1.11 = 99900
    expect(cart.calculateTotal(0.1, 0.11)).toBe(99900);
  });

  test('should throw error when discount out of range', () => {
    expect(() => cart.calculateTotal(1.5)).toThrow('Invalid discount rate');
  });

  test('should throw error when tax out of range', () => {
    expect(() => cart.calculateTotal(0, -0.1)).toThrow('Invalid tax rate');
  });
});

// ===== getItemCount =====
describe('getItemCount', () => {
  test('should return 0 when cart empty', () => {
    expect(cart.getItemCount()).toBe(0);
  });

  test('should sum all quantities', () => {
    cart.addItem({ name: 'A', price: 1, quantity: 3 });
    cart.addItem({ name: 'B', price: 1, quantity: 2 });
```

```
      expect(cart.getItemCount()).toBe(5);
    });
  });

  // ===== clear =====
  describe('clear', () => {
    test('should empty the cart', () => {
      cart.addItem({ name: 'A', price: 1 });
      cart.clear();
      expect(cart.items).toHaveLength(0);
    });
  });
});
```

Total: 15 test case ✅ (melebihi minimum 10)

Langkah 4.4 — Jalankan Test & Coverage

```
bash
npm run test:coverage
```

Output target:

```
text
PASS  tests/ShoppingCart.test.js
  ...
Tests:       15 passed, 15 total
```

```
File             | % Stmts | % Branch | % Funcs | % Lines
ShoppingCart.js  |   100   |   100    |   100   |   100
```

⚠ **Jika coverage < 80%:** tambahkan test untuk branch yang belum teruji. Cek laporan di `coverage/lcov-report/index.html` (buka di browser).

Langkah 4.5 — Cek Lint & Format

```bash
npm run lint
npm run format:check
```

Jika ada error format, perbaiki otomatis:

```bash
npx prettier --write .
npx eslint . --fix
```

Langkah 4.6 — Pastikan CI Hijau

```bash
git add .
git commit -m "feat(cart): add ShoppingCart with 15 unit tests"
git push
```

Cek tab **Actions** di GitHub → semua step harus hijau ✅.

Langkah 4.7 — Buat README

Buat `README.md` di root project:

````
markdown
# Standarisasi Coding — Tugas Pengujian & Integrasi

## 📋 Deskripsi
Project latihan penerapan standar pengujian (AAA, FIRST) dan integrasi kode
(GitHub Actions CI) pada modul ShoppingCart dan User.

## 🛠️ Teknologi
- Node.js v20
- Jest (testing)
- ESLint (linting)
- Prettier (formatting)
- GitHub Actions (CI)

## 🚀 Cara Menjalankan

```bash
npm install
npm test
npm run test:coverage
npm run lint
npm run format:check
````
````

📊 Coverage

https://img.shields.io/badge/coverage-100%25-brightgreen

🗂 Struktur

```
text
src/          → source code
tests/        → unit test
.github/      → CI workflow
```

👤 Author

- Nama: [Nama Anda]
- NIM: [NIM Anda]

📝 Catatan

- Semua test mengikuti struktur AAA
- Penamaan test: `should [behavior] when [condition]`
- Coverage target: 80%+ (tercapai: __%)

```
text

### Langkah 4.8 — Submit
```

```
Kirim **link repo GitHub** (public) ke dosen/portal kuliah.

**Format pengumpulan:**
```

Nama : [Nama]
NIM : [NIM]
Repo : https://github.com/USERNAME/standarisasi-coding
Coverage: __%
Status CI: ✅ Passing

```
text

### ✅ Checklist Tugas Mandiri

- [ ] Minimal **10 test case** struktur AAA (target: 15)
- [ ] Penamaan `should...when...` konsisten
- [ ] Semua test **passing**
- [ ] Coverage ≥70% (target 80%+)
- [ ] `npm run lint` dan `npm run format:check` lulus
- [ ] CI **hijau** di GitHub Actions
- [ ] `README.md` lengkap
- [ ] Commit message konvensional
- [ ] Repo **public** & link disubmit

---

## 🆘 Troubleshooting Umum
```

```
| Masalah | Penyebab | Solusi |
|---------|----------|--------|
| `npm ci` gagal di CI | `package-lock.json` tidak ada | `npm install` lokal → commit lockfile |
| Coverage < threshold | Branch belum teruji | Buka `coverage/lcov-report/index.html`, lihat baris merah |
| `jest: command not found` | Belum `npm install` | Jalankan `npm install` |
| CI merah di step lint | Format tidak sesuai Prettier | `npx prettier --write .` |
| `git push` ditolak | Remote belum diset | `git remote add origin <URL>` |
| Coverage tidak muncul di artifact | Path salah | Cek `path: coverage/` di `ci.yml` |
| Test flaky (kadang pass kadang fail) | Ada timing/order dependency | Hindari `setTimeout`, gunakan `beforeEach` reset state
|
| `module not found` | Path import salah | Cek `require('../src/...')` |

---

## 📊 Rubrik Penilaian (Referensi)

| Aspek | Bobot | Kriteria |
|-------|-------|----------|
| **Latihan 1** — Refactor test | 15% | Struktur AAA, penamaan, semua pass |
| **Latihan 2** — CI pipeline | 20% | CI hijau, step lengkap |
| **Latihan 3** — Analisis | 15% | Analisis mendalam, roadmap realistis |
| **Tugas: Jumlah test** | 15% | ≥10 test bermakna |
| **Tugas: Coverage** | 15% | ≥70% (nilai penuh ≥80%) |
| **Tugas: Kode & standar** | 10% | Lint, format, commit konvensional |
| **Tugas: Dokumentasi** | 10% | README lengkap & jelas |
```

```
---

## 🎯 Ringkasan Alur Pengerjaan
```

[Bagian 0] Setup (30m)
↓
[Latihan 1] Buat User.js + 9 test AAA (45m) → commit
↓
[Latihan 2] ci.yml + push ke GitHub (30m) → cek Actions hijau
↓
[Latihan 3] ANALISIS.md (30m) → commit
↓
[Tugas Mandiri] ShoppingCart + 15 test + README (2-3j) → submit

```
text

**Tips akhir:**
- 💡 Commit **sering & kecil** — bukan sekali besar
- 💡 **Baca error message** di CI sebelum tanya
- 💡 **Buka HTML coverage report** untuk tahu baris mana yang belum teruji
- 💡 Jika macet >30 menit di satu masalah → tanya dosen/asisten

---

*Selamat mengerjakan! Fokus pada proses standar, bukan hanya hasil akhir.*
```